JAKA Mini 2 机器人应用挑战赛

基础组仿真教程

没有设备时怎么学、怎么用

基础组可以在没有机械臂、相机和工件的学校先开起来。记分规则仍是 competition.md 里的「赛道一」：读一张二维码，按 6 个固定工位的顺序，把工件放进一只料盒。

仓库里已经有两条仿真。判分线用 MoveIt 假硬件，出题、发令和成绩都在 jaka_competition_kit。视觉线是另一套 Gazebo 世界，腕部有 Gemini 305，台上是 6 个几何体。round.launch.py 的判分器不读视觉线训出来的权重。

没有设备时，先在判分线上跑通自己的程序，再在视觉线上做采集、训练和推理。两套仿真同时开，后一套会连到前一套的 controller_manager，接着在 Failed to configure controller 里退出。重开一整套仿真之前，先跑 bash setup/sim-clean.sh。

仓库根目录的 基础组仿真教程.pdf 与本文同一份内容。打印和转发用 PDF。

没有机械臂和相机时，判分练习和视觉练习覆盖的范围不同。

项目	判分练习	视觉练习
练什么	读二维码，按顺序抓固定工位	采集、训练、推理，再按码去抓
认不认形状	不认类别	认 6 个几何体。赛道一记分不含类别
怎么开	round.launch.py track:=1	先预览采集，训完再 learn:=true
算不算成绩	算，写到 ~/.ros/jaka_competition/	不算。ref_track1 不读这个权重
限时	145 秒。条件是假硬件，且 vel_scale:=1.0	Gazebo 按关节速度走。用时不能拿去对 145 秒

```
bash setup/sim-clean.sh
```

ros2 launch 跑完一轮后进程还在。留下它们再开一套，就会撞上上面的控制器冲突。

背景：赛道一记什么分

二维码格式是 JAKA1- 加 6 位数字。这 6 位是工位号 1 到 6 的一个排列。JAKA1-246135 表示顺序 2、4、6、1、3、5。跳序或错序的那一件不计分，后面仍按码里的顺序继续。

工位靠近机械臂的一排，从 −x 到 +x 是 1、2、3，远排是 4、5、6。工件一开始在工位中心。料盒只有一只。二维码在 (0, 110) mm，半径小于手臂够得到的 180 mm。正式比赛用台面上方的俯拍相机读这张卡，手臂不到那里去。

满分 100。按顺序每放进料盒 1 件得 10 分，共 60 分。105 秒内做完再得 40 分效率分，之后每 5 秒扣 4 分。这组时间是假硬件、速度缩放 1.0 时标定的。

种子 246135 在本仓库的 make_order 下生成 JAKA1-165324，顺序是 1、6、5、3、2、4。解码结果和这串不一致时，先查解码，种子先保持 246135。

现状：两条仿真各接哪段代码

判分线已经接在这些位置：

1. round.launch.py 的默认 world 是 rviz，也就是 MoveIt 假硬件。
2. 官方参考实现是 ref_track1。它等 /competition/start 把状态变成 running 之后才动。

3. 规则书里的 145 秒和 105 秒，对应启动参数 vel_scale:=1.0。这个参数不写时默认是 0.5，用时会比规则书里的数字长。
4. 同一套参考实现，假硬件上的中位数是 102.7 秒。Gazebo 里关节要按速度走完，赛道一测到过 209.4 秒。判分只看 MoveIt 里的规划场景。

视觉线接在另一套世界上：

1. 入口是 gemini305_preview.launch.py。不加 learn:=true 时，它把手臂送到识别位姿，并摆上 6 个几何体。
2. learn:=true 才出题、解码、推理、抓取。
3. 这里的工位是预览世界里够得到的位置：近排 y=0.255 m，远排 y=0.338 m，x 为 −0.120、0、0.120 m。比赛那套 150 mm 网格不在这个世界里。

远程桌面里 Gazebo 窗口发黑或拖不动时，先看 docs/Gazebo仿真.md 第 4 节。那是显示会话的问题。判分线可以不看 Gazebo。

猜想：没有设备时可以先分开练

1. 记分、顺序、工位、料盒，放在判分线上练。这条线不需要相机。
2. 自己的节点替换 ref_track1，仍订同一套 /competition/scene 和 /competition/run_state。
3. 相机、6 类几何体、检测框和顶面抓取点，放在视觉线上练。
4. 进阶组 20 种日用品的外观不在这两条线里。仿真里那些物体是盒子和圆柱。

前面的限时只回答第 1 点。第 3 点训出的 best.pt 不进入第 1 点的成绩。

安装

每台电脑在仓库根目录做一次：

```
bash setup/install-deps.sh
bash setup/patch-moveit-launches.sh
bash setup/build.sh
```

不做 patch-moveit-launches.sh 时，use_rviz_sim:=true 会被静默忽略，假硬件模式不会生效。以后 apt 升级了 MoveIt，再执行一次。

jaka_ros2/build、install、log 里是编译那台机器的绝对路径。换电脑之后自己编译，不拷这三份目录。

每个新终端都要：

```
source /opt/ros/humble/setup.bash
source jaka_ros2/install/setup.bash
```

下面的命令都假设已经 source 过，并且当前目录是仓库根目录。

视觉线还要用系统里的这几个包。只跑判分线时可以先不装：

```
sudo apt install -y ros-humble-cv-bridge python3-opencv python3-qrcode python3-pyzbar libzbar0
```

判分线怎么跑完一轮

145 秒和 105 秒对应假硬件、vel_scale:=1.0。下面的命令用同一组参数。

终端 1：

```
bash setup/sim-clean.sh
ros2 launch jaka_competition_kit round.launch.py track:=1 seed:=246135 vel_scale:=1.0
```

要看 Gazebo 画面再加 world:=gazebo。成绩仍按规划场景算，不按 Gazebo 里的用时算。

等终端 1 里的控制器起来，终端 2：

```
source /opt/ros/humble/setup.bash
source jaka_ros2/install/setup.bash
ros2 service call /competition/generate std_srvs/srv/Trigger "{}"
ros2 service call /competition/start std_srvs/srv/Trigger "{}"
```

先 generate 再 start。参考实现等到 running 才动，计时从发令开始。

round.launch.py 带起来的 RViz 用的是 moveit.rviz，里面没有规划场景，看不到工作台和工件。赛场在另一个终端：

```
source /opt/ros/humble/setup.bash
source jaka_ros2/install/setup.bash
ros2 launch jaka_competition_kit competition_rviz.launch.py
```

这个 launch 只用 competition.rviz，可以和上面那场一起开，不再起一套控制器。

这一轮接上之后，能看到三件事：

1. 参考实现的终端打印 JAKA1-165324，然后按工位 1、6、5、3、2、4 去抓。
2. competition_rviz 里有工作台、6 块工位板、料盒和工件。
3. 成绩在 ~/.ros/jaka_competition/score_<时间戳>.txt，里面的顺序和这串码一致。

这一轮跑的是参考实现，用来对照计分。提交的程序不交 ref_track1。

换一场题，再调一次 generate，再 start。重开整套仿真之前先 bash setup/sim-clean.sh。

自己的程序接在哪两个话题上

能跑通的最小例子分在两个文件里：

- jaka_competition_kit/jaka_competition_kit/ref_track1.py 把二维码解成抓取顺序。
- jaka_competition_kit/jaka_competition_kit/reference.py 等题目、等发令，再按顺序调用抓取。

它订两个话题：

- /competition/scene 里是这一轮的工件。每件有 station、xy、grasp_tcp_z、approach_z、release_tcp_z。qr_png 是二维码图片的路径。
- /competition/run_state 变成 running 之后才动作。在这之前的动作不算成绩。

解码读 qr_png。qr_order 是出题节点写下的真值，正式程序不读它。工位用场景里的 xy，也就是 arena.yaml 里那套固定工位。

自己的节点这样接。终端 1 不再起参考实现：

```
bash setup/sim-clean.sh
ros2 launch jaka_competition_kit round.launch.py track:=1 seed:=246135 vel_scale:=1.0 run_reference:=false
```

终端 2 出题、发令。终端 3 跑自己的节点。抓取用 Executor.pick_and_place。某一步失败时走 recover()：工件还在夹爪上，就放回原位，并发 abort。判分器只把释放点不在料盒里的情况记成掉落。规划失败后把工件从场景里删掉，或直接放进料盒，后面重试成功仍可能被记一次掉落。

提交自己的功能包或 solve.py。判分器保持不动。

视觉线在补相机和工件

前面的猜想把类别识别放在这条线。赛道一的得分项里没有类别。

类别编号和 perception/track1_sim/shapes.py 一致。改了编号，旧权重会对不上。

id	名字	中文
0	cube	正方体
1	rod	长方体
2	cylinder	圆柱体
3	hex	六角柱
4	tri	三角柱
5	cone	圆台

二维码仍是 JAKA1- 加 6 位排列。这个世界没有俯拍相机，卡片放在远排垫板外侧，落在腕部画面下沿。比赛卡片在 (0, 110) mm，由俯拍相机读。

YOLO 给出类别和框。抓取点用框里面工件顶面的深度，反投影到世界坐标。检测框中心和工位名义坐标都不拿来当夹持点。

检测环境装在固定路径

grasp.py 和实时窗口调用 perception/track2_yolo/.venv/bin/python。只做基础组时，这个目录只装 YOLO，目录名保持不变，也不在这里做 20 类检测。ROS 节点是 Python 3.10，YOLO 不装进系统 Python。infer.py 单独跑推理，抓取节点用子进程调它。

perception/track2_yolo/requirements.txt 里的 PyTorch 按 RTX 5060 Ti、CUDA 13 锁定。显卡相同，按这个文件安装。显卡不同，先装能在自己机器上 import torch 的版本，再装 ultralytics==8.4.182。没有独显就装 CPU 版，训练时把 --batch 8 改成 --batch 4。

```
cd perception/track2_yolo
python3 -m venv .venv
.venv/bin/python -m pip install -U pip
.venv/bin/python -m pip install -r requirements.txt
```

有 uv 时可以用 Python 3.12：

```
cd perception/track2_yolo
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.txt
```

yolo11n.pt 是 COCO 预训练权重，还不会认这 6 个几何体。放到 perception/track2_yolo/weights/yolo11n.pt，文件名保持不变。train.py 发现文件不存在会退出，不会自己下载。学校网络访问 GitHub 不稳定时，用浏览器下好再拷进去。常被写成 YOLOv11 的权重，文件名是 yolo11n.pt，没有 yolov11n.pt。

采集

视觉线的相机在识别位姿。采集也在这个位姿下做，所以先开预览，不加 learn:=true，先不设 JAKA_GRIPPER。

```
bash setup/sim-clean.sh
ros2 launch jaka_competition_kit gemini305_preview.launch.py
```

等 Gazebo 里看得到腕部画面，另开一个终端：

```
source /opt/ros/humble/setup.bash
source jaka_ros2/install/setup.bash
python3 perception/track1_sim/capture_sim.py --samples 24
```

脚本把 6 件挪到两排格子里。每件沿 x 平移不超过 18 mm，沿 y 不超过 12 mm，再绕竖直轴转一个随机角。然后存彩色图和 YOLO 框，结束时放回演示位置。框来自仿真位姿，把几何体八个角投到图像上。有一件出了画面，这一摆不写入。

输出在 perception/track1_sim/dataset/incoming/<批次>/：

- 0001.jpg 旁边是 0001.txt。一行一个物体，格式是 类别 cx cy w h，数值是 0 到 1 的比例，类别是上表的 0 到 5。
- preview/ 里是画了框的图。训练会跳过这个目录。

打开 preview/，看框有没有包住物体。包不住的那张，连同旁边的 txt 一起删掉。

24 张够把这条流程走完。框没有包住物体就续采。--samples 是这一目录的总张数。目录里已经有 24 张时，写成 --samples 80 才会续到 80 张：

```
python3 perception/track1_sim/capture_sim.py \
 --samples 80 --resume \
 --out perception/track1_sim/dataset/incoming/<刚才的批次名>
```

图片和标注不进 git。预览没开，或者上一套仿真还占着相机，脚本会报等不到新的腕部图。

训练

预览可以关。训练不需要 ROS。

```
perception/track2_yolo/.venv/bin/python perception/track1_sim/train.py
```

脚本收集 dataset/incoming/ 下所有批次里、框文件非空的 jpg，打乱后大约 20% 做验证，其余做训练。仿真里每一张都是独立摆放，可以按张划分。少于 4 张会直接退出。

默认 30 个 epoch，输入 640，batch 8。没有 GPU：

```
perception/track2_yolo/.venv/bin/python perception/track1_sim/train.py --batch 4
```

训练关掉了 AMP 自检，避免临时去下载另一个权重。权重写到：

```
perception/track1_sim/runs/track1/weights/best.pt
```

再训一次会覆盖这个目录。best.pt 不进 git。换电脑时自己拷过去，或者在那台电脑上重新采、重新训。

推理

这一步只看一张已经采过的图，手臂不动。它用来确认权重认的是上面 6 个几何体。

```
perception/track2_yolo/.venv/bin/python perception/track1_sim/infer.py \
 perception/track1_sim/dataset/incoming/<批次>/0001.jpg
```

成功时标准输出只有一行，其余日志在标准错误里。id 是类别名字，xyxy 是像素框：

```
YOLO_JSON={"detections":[{"id":"cube","conf":0.91,"xyxy":[120.0,80.0,210.0,190.0]}]}
```

没有 best.pt 会直接退出。yolo11n.pt 不拿来认这 6 类。置信度门槛默认 0.40，要更严的框可以加 --conf 0.6。

验证集的框可以画出来：

```
perception/track2_yolo/.venv/bin/python -c '
from ultralytics import YOLO
m = YOLO("perception/track1_sim/runs/track1/weights/best.pt")
m.predict(
 source="perception/track1_sim/dataset/images/val",
 save=True,
 project="perception/track1_sim/runs",
 name="pred",
)
'
```

画好的图在 perception/track1_sim/runs/pred/。

按码抓取

夹持高度按柔性爪指尖计算。不设 JAKA_GRIPPER=1 时，高度按裸法兰算，指尖会低于网格。预览那场先关掉。

```
bash setup/sim-clean.sh
source /opt/ros/humble/setup.bash
source jaka_ros2/install/setup.bash
export JAKA_GRIPPER=1
ros2 launch jaka_competition_kit gemini305_preview.launch.py learn:=true
```

这条命令会做完这些事：

1. 打开带腕部相机和 6 个几何体的 Gazebo，手臂到识别位姿。
2. 6 个几何体换到 6 个工位，并摆上这一轮的 JAKA1-######。
3. 再起一个 move_group，接到 Gazebo 里已经有的控制器，不再起一套假硬件。
4. 用腕部画面解码。解不出来就停下，生成时的那串数字不会被拿来代替。
5. 用 best.pt 推理。按码的顺序，把框里的顶面深度换成世界坐标，再放进画面外的蓝色练习料盒。
6. Gazebo 里的网格不跟着规划场景走。脚本另发位姿，让它跟着夹爪，放进料盒后停住。

运行时能看到：

- Gazebo 里，工件按二维码顺序离开垫板，进入右前方的蓝色料盒。
- 「基础组识别」窗口是腕部实时画面。底栏先是正在读码，解出来以后是顺序。绿框是检测，红叉是顶面抓取点。关掉窗口或按 q 结束。
- 定格图在 perception/track1_sim/runs/grasp/detect.jpg，读码用的原图是同目录的 frame.jpg。
- 话题 /track1/yolo 是这一帧的 JSON，里面有 order 和每件的 station。

换权重时，仿真保持开着，另开终端再跑一次。手臂会先回到识别位姿：

```
source /opt/ros/humble/setup.bash
source jaka_ros2/install/setup.bash
export JAKA_GRIPPER=1
python3 perception/track1_sim/grasp.py --ros-args -p use_sim_time:=true \
 -p weights:=perception/track1_sim/runs/track1/weights/best.pt
```

实时窗口开不了时，先确认 perception/track2_yolo/.venv/bin/python 存在。没有这个解释器，推理不会启动，窗口也不会出现。

影响面

这条练习留下的边界是：

- best.pt 只覆盖仿真里的 6 个几何体。赛道一记分不读它。
- 进阶组 20 种日用品仍要等真机腕部相机，按 perception/track2_yolo/ 采集。这个权重不覆盖那些外观。
- 视觉线的工位坐标不能写进判分线的抓取程序。判分线用工位来自 arena.yaml。
- Gazebo 里的用时不能拿去对 145 秒和 105 秒。

观测：怎样确认这一步已经接上

按前面四条猜想核对：

1. 判分线，种子 246135 解出 JAKA1-165324。6 件按这个顺序进料盒，成绩文件里的顺序和它一致。
2. 替换参考实现之后，run_reference:=false 仍能等到 running，并读 qr_png，不读 qr_order。
3. 视觉线里，preview/ 的框包住几何体。infer.py 打出的 id 只使用 cube、rod、cylinder、hex、tri、cone。
4. learn:=true 解不出码就停。best.pt 不作为赛道一的提交，也不拿去认进阶组的实物。